



Web *scraping* con Python y BeautifulSoup

La acción de extraer datos nos plantea un interesante punto de partida para explorar y comprender diversos aspectos clave en el manejo de información.

Primero, **la diversidad de formatos y estructuras** que se pueden presentar al momento de querer encontrar información, ya que cada sitio puede exponer su información de una forma muy distinta a la de otros. Por otro lado, **el problema de la disponibilidad**, ya que la fuente de los datos puede cambiar su formato en cualquier momento, no hay nada que nos garantice que nuestros *scripts* para obtener información sigan funcionando a lo largo del tiempo, incluso, que el sitio de donde estamos extrayendo la información siga activo.

Python es actualmente uno de los lenguajes de programación más utilizados debido a su versatilidad y potencia. Una de sus mayores ventajas es su amplio ecosistema de librerías especializadas para diversas tareas.

En el contexto del **web scraping** (extracción de datos de páginas web), dos herramientas destacadas son:

- **Beautiful Soup**: Una librería diseñada para analizar y extraer información de archivos HTML y XML de manera eficiente.
- **Requests**: Una librería ampliamente usada para realizar peticiones HTTP, permitiendo recuperar el contenido de páginas web antes de procesarlo.

La combinación de estas dos librerías permite automatizar la recolección de datos desde fuentes en línea, facilitando su posterior análisis y almacenamiento.

Web scraping

¿Por qué usar *web scraping*? La respuesta es muy sencilla, **porque los sitios web tienen una gran variedad de formatos, estilos, estructuras y maneras de organizar la información, además de que estas estructuras pueden ser cambiadas por parte de quien hace el sitio**, es como cuando vas a un local comercial o a un supermercado, y acomodan los productos de manera diferente, o le cambian la imagen o colores.

La *web* es el conjunto de documentos accesibles a través de Internet que tienen una **estructura que puede ser entendida por los navegadores**, por ejemplo; Microsoft Edge, Mozilla Firefox, Safari, Google Chrome, entre otros. Esta estructura tiene un **formato conocido como HTML**, que es un lenguaje de marcado de Hipertexto el cual abordaremos más adelante.

La *web* utiliza un **protocolo** para que las entidades que solicitan y envían información puedan entenderse. Este protocolo es el **Protocolo de Transferencia de Hiper Texto (HTTP - HyperText Transfer Protocol)**.

Cuando abrimos un navegador y queremos ver el contenido de alguna página web, escribimos una dirección en el navegador, esta se traduce a una **dirección IP** que definiremos en otras palabras, **como una dirección que utiliza el protocolo de Internet**, esta dirección es la de un servidor en algún lugar del mundo que aloja eso que quieres consultar.

En este sentido, **se utiliza el protocolo HTTP para hacer la solicitud, y el servidor responde con información en formato HTML** que el navegador sabe exactamente cómo desplegar en tu pantalla.

Haciendo web scraping

Pongamos manos a la obra, veamos cómo hacer *scraping* usando un ejemplo:

Supongamos que te interesa revisar cuáles son las versiones que puedes descargar del sistema operativo *Linux Mint* desde su página oficial, mismo que te invitamos a visitar al termino de este recurso en el material adicional.

Este sitio nos muestra la información que necesitamos, aunada a otra información, la tabla que nos interesa se ve así:

Version	Codename	Edition	Package base	Status
22.1	Xia	Cinnamon	Ubuntu Noble	Long term support release (LTS), supported until April 2029.
		MATE		
		Xfce		
22	Wilma	Cinnamon	Ubuntu Noble	Long term support release (LTS), supported until April 2029.
		MATE		
		Xfce		
21.3	Virginia	Cinnamon	Ubuntu Jammy	Long term support release (LTS), supported until April 2027.
		Cinnamon (Edge ISO)		
		MATE		
		Xfce		
21.2	Victoria	Cinnamon	Ubuntu Jammy	Long term support release (LTS), supported until April 2027.
		Cinnamon (Edge ISO)		
		MATE		
		Xfce		

21.1	Vera	Cinnamon	Ubuntu Jammy	Long term support release (LTS), supported until April 2027.
		MATE		
		Xfce		
21	Vanessa	Cinnamon	Ubuntu Jammy	Long term support release (LTS), supported until April 2027.
		MATE		
		Xfce		
20.3	Una	Cinnamon	Ubuntu Focal	Long term support release (LTS), supported until April 2025.
		Cinnamon (Edge ISO)		
		MATE		
		Xfce		
20.2	Uma	Cinnamon	Ubuntu Focal	Long term support release (LTS), supported until April 2025.
		Cinnamon (Edge ISO)		
		MATE		
		Xfce		
20.1	Ulyssa	Cinnamon	Ubuntu Focal	Long term support release (LTS), supported until April 2025.
		Cinnamon (Edge ISO)		
		MATE		
		Xfce		
20	Ulyana	Cinnamon	Ubuntu Focal	Long term support release (LTS), supported until April 2025.
		Cinnamon (Edge ISO)		
		MATE		
		Xfce		
6	Faye	Cinnamon	Debian Bookworm	Long term support release (LTS)

Lo que tenemos que hacer, es **conocer la estructura HTML de este documento**, misma que podemos ver dando clic derecho en nuestro navegador y seleccionar la opción “ver código fuente”, o podemos presionar la tecla F12 para abrir la vista de “inspeccionar elemento” de nuestro navegador, o bien, podemos descargar la página para examinar el código HTML.

Después de acomodar el indentado, la estructura HTML de esta tabla se ve de la siguiente manera:

```
<table class="table table-lg table-bordered text-left bg-light">
<thead class="all-text-white bg-grad">
  <tr class="text-uppercase">
    <th scope="col">Version</th>
    <th scope="col">Codename</th>
    <th scope="col">Edition</th>
    <th scope="col">Package base</th>
    <th scope="col">Status</th>
  </tr>
</thead>
<tbody>

  <tr>
```

```

<td rowspan="3">22.1</td>

<td rowspan="3">Xia</td>

<td><a href="edition.php?id=319">Cinnamon </a></td>

<td rowspan="3">Ubuntu Noble</td>

<td rowspan="3">Long term support release (LTS),
supported until April 2029.</td>

</tr>

<tr>

<td><a href="edition.php?id=321">MATE </a></td>

</tr>

<tr>

<td><a href="edition.php?id=320">Xfce </a></td>

</tr>

<tr>

<td rowspan="3">22</td>

```

```
<td rowspan="3">Wilma</td>

<td><a href="edition.php?id=316">Cinnamon </a></td>

<td rowspan="3">Ubuntu Noble</td>

<td rowspan="3">Long term support release (LTS),
supported until April 2029.</td>

</tr>

<tr>

<td><a href="edition.php?id=318">MATE </a></td>

</tr>

<tr>

<td><a href="edition.php?id=317">Xfce </a></td>

</tr>
```

...

En este caso, podemos ver el código HTML, en donde apreciamos que la tabla tiene una clase `class="table table-lg table-bordered text-left bg-light"`, esta información puede sernos de bastante utilidad cuando queramos extraer solamente la información de esta tabla.

Importante: Si aún no te sientes familiarizado con el formato de archivos HTML; te recomendamos que te tomes un tiempo para analizar varios documentos de tal forma que te sientas más cómodo con el formato de este tipo de archivos.

Seguramente ya estás ansioso de trabajar con Python, y ver cómo podemos extraer esta información, veamos cómo hacerlo con la ayuda de la librería **Beautiful Soup**.

Como ya sabes, para poder utilizar una librería en Python, primero hay que instalarla. En Google Colab instala las librerías `requests` y `beautifulsoup4`, empleando las siguientes instrucciones:

```
!pip install requests
!pip install beautifulsoup4
```

Estas dos líneas instalan las librerías necesarias. Usan `!` para indicar que son comandos del sistema (no de Python), y `pip` es el instalador de paquetes de Python.

Si estás trabajando en tu computadora, te recomendamos utilizar ambientes virtuales de Python. En la sección de material adicional te dejamos algunas referencias.

La primera librería, `requests`, la utilizaremos para comunicar Python con la *web*, mientras que `beautifulsoup4` la utilizaremos para analizar el contenido HTML que nos regresa la petición realizada.

Para poder utilizar las librerías que acabamos de instalar, hay que importarlas en nuestro *script*, esto lo hacemos con las siguientes instrucciones.

```
import requests
from bs4 import BeautifulSoup
```

Lo que haremos a continuación es hacer la solicitud al sitio, es decir, haremos una petición usando el protocolo HTTP, en este caso utilizaremos el sitio con el que estamos trabajando de las versiones del sistema operativo *Linux Mint* que aún tienen el soporte que revisamos anteriormente. En una variable guardamos la URL del sitio del que queremos extraer la información.

```
URL = "https://linuxmint.com/download_all.php"
```

Enseguida, hacemos una solicitud para leer el contenido de dicha URL, y almacenamos el resultado en una variable, en este caso la llamamos **page**.

```
page = requests.get(URL)
```

Ahora sí, haremos uso de la librería `BeautifulSoup` para analizar el contenido HTML de lo que tenemos almacenado en la variable **page**.

```
soup = BeautifulSoup(page.content, "html.parser")
```

Esto nos crea una estructura en la que ya podemos buscar por elementos HTML específicos, en nuestro caso, analizando el código HTML de la tabla que queríamos analizar, nos damos cuenta de que la tabla está en un elemento llamado `<table>`, con atributo `class="table table-lg table-bordered text-left bg-light"`:

```
<table class="table table-lg table-bordered text-left bg-light">
```

Podemos pedirle a BeautifulSoup que nos busque ese elemento en nuestro HTML, esto mediante la siguiente instrucción:

```
table = soup.find('table', attrs={ "class" : "table table-lg  
table-bordered text-left bg-light"})
```

Aquí, a nuestro objeto `soup`, que es donde teníamos nuestra estructura HTML, le pedimos que busque el elemento `"table"` que tenga como atributos `class="table table-lg table-bordered text-left bg-light"`

Si imprimimos el contenido de la variable `table`, vemos que almacena todo el elemento `"table"` que nos interesa.

```
print(table)
```

```
<table class="table table-lg table-bordered text-left bg-li-  
ght">
```

```
<thead class="all-text-white bg-grad">
```

```
<tr class="text-uppercase">
```

```
<th scope="col">Version</th>
```

```
<th scope="col">Codename</th>
```

```

<th scope="col">Edition</th>

<th scope="col">Package base</th>

<th scope="col">Status</th>

</tr>

</thead>

<tbody>

<tr>

<td rowspan="3">22.1</td>

<td rowspan="3">Xia</td>

<td><a href="edition.php?id=319">Cinnamon </a></td>

<td rowspan="3">Ubuntu Noble</td>

<td rowspan="3">Long term support release (LTS), supported
until April 2029.</td>

</tr>

.
.
.

<tr>

```

```

<td><a href="edition.php?id=282">MATE </a></td>

</tr>

<tr>

<td><a href="edition.php?id=283">Xfce </a></td>

</tr>

<tr>

<td rowspan="1">6</td>

<td rowspan="1">Faye</td>

<td><a href="edition.php?id=308">Cinnamon </a></td>

<td rowspan="1">Debian Bookworm</td>

<td rowspan="1">Long term support release (LTS).</td>

</tr>

</tbody>

</table>

```

Nota: el código fue recortado para fines prácticos de esta demostración.

Ahora que tenemos toda la información de la tabla en la variable `table`, podemos pedirle a `BeautifulSoup` que nos dé todos los elementos `<tr>` (*table row*, filas de la tabla) de esa tabla, porque en cada renglón hay información sobre cada una de las versiones de dicho Sistema Operativo (S. O) con la instrucción:

```
t1 = table.find_all("tr")
```

Obtenemos el conjunto de todos los renglones de la lista.

Aquí podemos ver que el primer renglón de la tabla es diferente a los demás, porque este primer renglón tiene los encabezados.

Entonces, podríamos recorrer el primer renglón para extraer cada uno de los encabezados de la tabla de la siguiente manera:

```
tds = t1[0].find_all("th")
```

```
for y in tds :
```

```
    #print(y.text.rstrip("\n"))
```

```
    print(y.text)
```

```
print()
```

La primera instrucción (`tds = t1[0].find_all("th")`), buscará los elementos `<th>` en el primer reglón de la tabla, es decir, los encabezados y posteriormente guardarlos en la variable `tds`.

Luego en el ciclo `for`, iteramos en esa lista de elementos `<th>` para ir extrayendo cada uno de los valores. Cuando imprimimos `y.text` estamos mandando a imprimir los encabezados de cada columna.

El resultado de ejecutar estas instrucciones es:

```
Version
Codename
Edition
Package base
Status
```

Lo anterior es una lista de los encabezados que más adelante podríamos utilizar por ejemplo para ponerle nombre a las columnas de un *DataFrame*.

Bien, ya tenemos el encabezado, solo procesamos el primer elemento `<tr>`, ahora procesemos todos los demás, pero sin tomar en cuenta al primero.

Lo que haremos con cada renglón, es encontrar todos los elementos `<td>` y almacenarlos en la variable `tds`

Ya que tengamos la variable `tds`, vamos a construir una cadena de texto que almacene toda la información relativa a cada versión de *Linux Mint* en la tabla. En este ejemplo podemos ver que nuestra tabla está construida de manera interesante, por cada versión del S.O. tenemos tres “sabores”: *Cinnamon*, *Mate* y *Xfce*, así que solo queremos tomar en cuenta los elementos `<tr>` que NO inicien con estos nombres.

Para mandar a imprimir la información de cada versión, construiremos una cadena que contenga todos los campos de la fila de la tabla, esto lo podemos hacer con las siguientes instrucciones:

```
# recorreremos los renglones de la tabla, iniciando el segundo, el primero tiene encabezados
```

```
for x in range(1,len(t1)):
```

```
    # para cada fila, busquemos todos los elementos <td> y guardemos el resultado en la variable tds
```

```
    tds = t1[x].find_all("td")
```

```
    # inicialicemos una cadena en donde guardaremos la información de cada version
```

```
    str = ""
```

```
    # recorramos los elementos <td> de cada fila de la tabla,
```

```
    # y si el texto del elemento <td> no inicia con un "MATE", "Xfce" o "Cinnamon"
```

```
    # agregamos ese texto a la cadena que estamos construyendo,
```

```
    # agregando al final los caracteres " - " para separar los campos
```

```
    for y in tds :
```



```
if not(y.text.startswith("MATE") or y.text.startswi-
th("Xfce") or y.text.startswith("Cinnamon")):

    str += y.text.strip() + " - "

# finalmente mandamos a imprimir la cadena que construimos,

# no sin antes quitarle espacios y verificar que no nos
queda

# una cadena vacía

if str.strip() != "" :

    print(str[:-3])

# Imprimios str[:-3] para quitar los ultimos 3 caracteres
que usabamos como separador entre campos
```

El resultado de ejecutar estas líneas es:

22.1 - Xia - Ubuntu Noble - Long term support release (LTS),
supported until April 2029.

22 - Wilma - Ubuntu Noble - Long term support release (LTS),
supported until April 2029.

21.3 - Virginia - Ubuntu Jammy - Long term support release
(LTS), supported until April 2027.

21.2 - Victoria - Ubuntu Jammy - Long term support release (LTS), supported until April 2027.

21.1 - Vera - Ubuntu Jammy - Long term support release (LTS), supported until April 2027.

21 - Vanessa - Ubuntu Jammy - Long term support release (LTS), supported until April 2027.

20.3 - Una - Ubuntu Focal - Long term support release (LTS), supported until April 2025.

20.2 - Uma - Ubuntu Focal - Long term support release (LTS), supported until April 2025.

20.1 - Ulyssa - Ubuntu Focal - Long term support release (LTS), supported until April 2025.

20 - Ulyana - Ubuntu Focal - Long term support release (LTS), supported until April 2025.

6 - Faye - Debian Bookworm - Long term support release (LTS).

De esta manera, estamos extrayendo la información de cada campo de la tabla. Esto es solo un ejemplo de lo que se puede hacer con BeautifulSoup. En este caso, utilizamos la función `find` para encontrar el elemento `<table>` que tuviera ciertos atributos, es decir:

```
soup.find('table', attrs={ "class" : "table table-lg table-bordered text-left bg-light"})
```

También utilizamos la función `find_all` para encontrar una lista de todos los elementos "`<tr>`" contenidos en nuestra estructura de tabla.

```
t1 = table.find_all("tr")
```

De una manera similar, también podemos buscar elementos por **id**, en muchos archivos HTML, los elementos tienen asignados un id, por ejemplo, tendríamos código HTML parecido a esto:

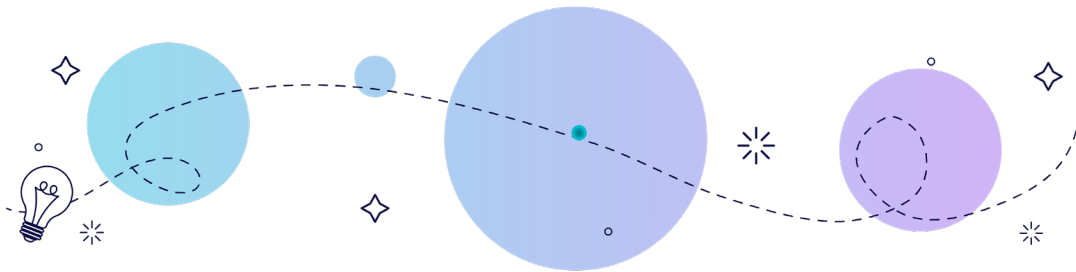
```
<div id="SomeContainer">
```

```
    <!-- aquí estaría la información que nos interesa -->
```

```
</div>
```

Aquí podríamos utilizar la función `find`, pasando como parámetro el id que queremos encontrar.

```
results = soup.find(id="SomeContainer")
```



■ Lista de episodios de la serie “Titans”

Ahora, analizaremos otro ejemplo, tomando como base los episodios de la serie de televisión “Titans”, misma que finalizó su emisión en 2023. En las siguientes imágenes podrás observar la lista de episodios separados por temporadas, esta información fue consultada desde Wikipedia (2025), podrás acceder a la URL desde el material adicional al finalizar esta lectura.

Entre otras cosas, podemos encontrar la lista de episodios separados por temporadas, que se ve más o menos así:

Season 1 (2018)

Main article: *Titans season 1*

No. overall	No. in season	Title	Directed by	Written by	Original release date	Prod. code
1	1	"Titans"	Brad Anderson	Story by: Akiva Goldsman & Geoff Johns & Greg Berlanti Teleplay by: Akiva Goldsman & Geoff Johns	October 12, 2018	T15.10146
2	2	"Hawk and Dove"	Brad Anderson	Akiva Goldsman	October 19, 2018	T13.20902
3	3	"Origins"	Kevin Rodney Sullivan	Richard Hatem & Geoff Johns and Marisha Mukerjee & Greg Walker	October 26, 2018	T13.20904
4	4	"Doom Patrol"	John Fawcett	Geoff Johns	November 2, 2018	T13.20905
5	5	"Together"	Meera Menon	Bryan Edward Hill & Gabrielle Stanton	November 9, 2018	T13.20906
6	6	"Jason Todd"	Carol Banker	Richard Hatem & Jeffrey David Thomas	November 16, 2018	T13.20907
7	7	"Asylum"	Alex Kalymnios	Bryan Edward Hill & Greg Walker	November 23, 2018	T13.20908
8	8	"Donna Troy"	David Frazee	Richard Hatem & Marisha Mukerjee	November 30, 2018	T13.20909
9	9	"Hank and Dawn"	Akiva Goldsman	Geoff Johns	December 7, 2018	T13.20910
10	10	"Koriand'r"	Maja Vrvilo	Gabrielle Stanton	December 14, 2018	T13.20911
11	11	"Dick Grayson"	Glen Winter	Richard Hatem	December 21, 2018	T13.20912

Season 2 (2019)

Main article: *Titans season 2*

No. overall	No. in season	Title	Directed by	Written by	Original release date	Prod. code
12	1	"Trigon"	Carol Banker	Akiva Goldsman & Geoff Johns & Greg Walker	September 6, 2019	T13.21651
13	2	"Rose"	Nathan Hope	Richard Hatem	September 13, 2019	T13.21652
14	3	"Ghosts"	Kevin Tancharoen	Tom Pabst	September 20, 2019	T13.21653
15	4	"Aqualad"	Glen Winter	Jamie Gorenberg	September 27, 2019	T13.21654
16	5	"Deathstroke"	Nick Gomez	Bianca Sams	October 4, 2019	T13.21655
17	6	"Conner"	Alex Kalymnios	Richard Hatem	October 11, 2019	T13.21656
18	7	"Bruce Wayne"	Akiva Goldsman	Bryan Edward Hill	October 18, 2019	T13.21657
19	8	"Jericho"	Toa Fraser	Kate McCarthy	October 25, 2019	T13.21658
20	9	"Atonement"	Boris Mojsovski	Jeffrey David Thomas	November 1, 2019	T13.21659
21	10	"Fallen"	Kevin Sullivan	Jamie Gorenberg	November 8, 2019	T13.21660
22	11	"E.L._O."	Millicent Shelton	Bianca Sams	November 15, 2019	T13.21661
23	12	"Faux Hawk"	Larnell Stovall	Tom Pabst	November 22, 2019	T13.21662
24	13	"Nightwing"	Carol Banker	Richard Hatem & Greg Walker	November 29, 2019	T13.21663

Season 3 (2021)

Main article: *Titans season 3*

No. overall	No. in season	Title	Directed by	Written by	Original release date	Prod. code
25	1	"Barbara Gordon"	Carol Banker	Richard Hatem & Geoff Johns	August 12, 2021	T13.22451
26	2	"Red Hood"	Carol Banker	Tom Pabst	August 12, 2021	T13.22452
27	3	"Hank & Dove"	Millicent Shelton	Jamie Gorenberg	August 12, 2021	T13.22453
28	4	"Blackfire"	Millicent Shelton	Stephanie Coggins	August 19, 2021	T13.22454
29	5	"Lazarus"	Boris Mojsovski	Bryan Edward Hill	August 26, 2021	T13.22455
30	6	"Lady Vic"	Nick Gomez	Joshua Levy & Prathi Srinivasan	September 2, 2021	T13.22456
31	7	"51%"	Nick Gomez	Kate McCarthy	September 9, 2021	T13.22457
32	8	"Home"	Larnell Stovall	Tom Pabst	September 16, 2021	T13.22458
33	9	"Souls"	Boris Mojsovski	Richard Hatem	September 23, 2021	T13.22459
34	10	"Troubled Water"	Larnell Stovall	Melissa Brides	September 30, 2021	T13.22460
35	11	"The Call Is Coming from Inside the House"	Carol Banker	Stephanie Coggins	October 7, 2021	T13.22461
36	12	"Prodigal"	Carol Banker	Jamie Gorenberg & Bryan Edward Hill	October 14, 2021	T13.22462
37	13	"Purple Rain"	Chad Lowe	Richard Hatem & Greg Walker	October 21, 2021	T13.22463

Season 4 (2022- 23)

Main article: Titans season 4

No. overall	No. in season	Title	Directed by	Written by	Original release date	Prod. code
				Part 1		
38	1	"Lex Luthor"	Nick Copus	Richard Hatem	November 3, 2022	T13.23551
39	2	"Mother Mayhem"	Nick Copus	Bryan Edward Hill	November 3, 2022	T13.23552
40	3	"Jinx"	Boris Mojsovski	Jamie Gorenberg	November 10, 2022	T13.23553
41	4	"Super Super Mart"	Boris Mojsovski	Tom Pabst	November 17, 2022	T13.23554
42	5	"Inside Man"	Jen McGowan	Joshua Levy & Prathi Srinivasan	November 24, 2022	T13.23555
43	6	"Brother Blood"	Jen McGowan	Richard Hatem	December 1, 2022	T13.23556
				Part 2		
44	7	"Caul's Folly"	Greg Walker	Melissa Brides	April 13, 2023	T13.23558
45	8	"Dick & Carol & Ted & Kory"	Greg Walker	Tom Pabst	April 13, 2023	T13.23559
46	9	"Dude, Where's My Gar?"	Eric Dean Seaton	Teleplay: Geoff JohnsStory by: Geoff Johns & Ryan Potter	April 20, 2023	T13.23557
47	10	"Game Over"	Jesse Warn	Bryan Edward Hill	April 27, 2023	T13.23560
48	11	"Project Starfire"	Nick Copus	Jamie Gorenberg	May 4, 2023	T13.23561
49	12	"Titans Forever"	Nick Copus	Richard Hatem	May 11, 2023	T13.23562

Supongamos ahora que queremos una lista de los nombres de cada episodio junto con su número de episodio en general, no el número de episodio dentro de la temporada. Analizando el código HTML, podemos ver que la estructura de los datos que nos interesan tiene una forma como esta:

```
<html>

<body>

<table class="wikitable plainrowheaders wikiepisodetable"
style="width:100%">

<tbody>
```

```

<tr style="color:white;text-align:center">

    <th scope="col"
style="background:#2C2961;width:5%;color:white"><abbr
title="Number">No.</abbr><br />overall</th>

    <th scope="col"
style="background:#2C2961;width:5%;color:white"><abbr
title="Number">No.</abbr> in<br />season</th>

    <th scope="col"
style="background:#2C2961;width:18%;color:white">Title</th>

    <th scope="col"
style="background:#2C2961;width:15%;color:white">Directed
by</th>

    <th scope="col"
style="background:#2C2961;width:34%;color:white">Written
by</th>

    <th scope="col"
style="background:#2C2961;width:15%;color:white">Original
release date</th>

    <th scope="col"
style="background:#2C2961;width:8%;color:white"><abbr
title="Production">Prod.</abbr><br />code</th>

</tr>

```

```
<tr class="vevent module-episode-list-row"
style="text-align:center;background:inherit;color:inherit">
```

```
<th scope="row" rowspan="1" id="ep1"
style="text-align:center">1</th>
```

```
<td style="text-align:center">1</td>
```

```
<td class="summary" rowspan="1" style="text-align:left">"Titans"</td>
```

```
<td style="text-align:center"><a href="/
wiki/Brad_Anderson_(director)" title="Brad Anderson
(director)">Brad Anderson</a></td>
```

```
<td style="text-align:center"><i>Story by</i>
<i>#8202;;: <a href="/wiki/Akiva_Goldsman" title="Akiva
Goldsman">Akiva Goldsman</a> & <a href="/wiki/Geoff_
Johns" title="Geoff Johns">Geoff Johns</a> & <a href="/
wiki/Greg_Berlanti" title="Greg Berlanti">Greg Berlanti</
a><br /><i>Teleplay by</i>#8202;;: Akiva Goldsman & Geoff
Johns</td><td style="text-align:center">October#160;12,#16
0;2018<span style="display: none;">#160;(<span class="bday
dtstart published updated itvstart">2018-10-12</span></
span></td><td id="pcT15.10146" rowspan="1" style="text-align:center">T15.10146</td>
```

```
</tr>
```



```

<tr class="vevent module-episode-list-row"
style="text-align:center;background:inherit;color:inherit">

    <th scope="row" rowspan="1" id="ep2"
style="text-align:center">2</th>

    <td style="text-align:center">2</td><td
class="summary" rowspan="1" style="text-align:left">"Hawk
and Dove"</td>

    <td style="text-align:center">Brad Anderson</td>

    <td style="text-align:center">Akiva Goldsman</
td>

    <td style="text-align:center">October&#160;
19,&#160;2018<span style="display: none;">&#160;(<span
class="bday dtstart published updated itvstart">2018-10-19</
span></span></td><td id="pcT13.20902" rowspan="1"
style="text-align:center">T13.20902</td>

</tr>

```

En este caso, vemos que el número de episodio está contenido en un elemento `<th>` que tiene como id `epX`, donde la `X` es el número de episodio que nos interesa. Aquí podemos extraer la lista de los números de episodio utilizando una expresión regular. Si no estás familiarizado con el uso de expresiones regulares, en la sección de material adicional te dejamos unas referencias para que leas más al respecto.

El nombre de cada episodio está contenido en un elemento `<td>`, pero acá tenemos la ventaja de que tiene asociada la clase `"summary"` en estos elementos.

Veamos cómo quedaría el código para extraer estas listas tanto de número de episodio como la lista de nombres de los mismos.

```
import requests

from bs4 import BeautifulSoup

import re

URL = "https://en.wikipedia.org/wiki/Titans_(2018_TV_series)"

page = requests.get(URL)

soup = BeautifulSoup(page.content, "html.parser")
```

Las primeras dos líneas importan tanto `requests` como `BeautifulSoup`, ya las trabajamos previamente. La tercera línea importa la biblioteca para que podamos trabajar con expresiones regulares. Las siguientes tres líneas definen la URL del sitio del que extraeremos información, hacen la petición y finalmente creamos el objeto `BeautifulSoup` para que podamos extraer la información. Son muy parecidas al ejemplo anterior.

```
ids = soup.find_all('th', id=re.compile("^ep\d+"))

idList = []

for id in ids:

    idList.append(id.text)
```

A continuación, buscamos todos los elementos `th`, donde el id inicie con “**ep**” y contenga uno o más dígitos, la parte “`\d`” quiere decir que siga un dígito, pero como tenemos episodios con dos dígitos agregamos el “`+`” que significa, en este caso, uno o más dígitos. Entonces nuestra expresión dice algo así como: “dame todos los elementos `<th>` que tengan como **id** algo que inicie con **ep** seguido de **uno o más dígitos**”. De esta forma, nos regresará una lista con los elementos `<th>` con ids = `ep1`, `ep2`, ... `ep48` y `ep49`

La siguiente línea crea una lista vacía en la que iremos agregando el texto de los números de episodio, mientras que las últimas dos de este fragmento de código son un ciclo que irá agregando a la lista el texto de cada uno de esos elementos `<th>`.

```
episodes = soup.find_all("td", class_="summary")
episodeList = []
for episode in episodes:
    episodeList.append(episode.text)
```

Esta tercera parte del código es parecida a la anterior, pero aquí buscamos todos los elementos `<td>` que tengan la clase “`summary`”, que contienen los nombres de los episodios. Las siguientes tres líneas son similares al código anterior, solo creamos una lista con los nombres de cada episodio.

Hasta este momento tenemos dos listas creadas; una con los números de episodio, y la otra con los nombres. Si queremos imprimirlos basta con mandarlos a pantalla con otro ciclo:

```
for i in range (len(idList)) :

    print(idList[i], episodeList[i])
```

Obteniendo como resultado, lo que queríamos, una lista de número y nombre de episodio:

- 1 "Titans"
- 2 "Hawk and Dove"
- 3 "Origins"
- 4 "Doom Patrol"
- 5 "Together"
- 6 "Jason Todd"
- 7 "Asylum"
- 8 "Donna Troy"
- 9 "Hank and Dawn"
- 10 "Koriand'r"
- 11 "Dick Grayson"
- 12 "Trigon"
- 13 "Rose"
- 14 "Ghosts"
- 15 "Aqualad"
- 16 "Deathstroke"
- 17 "Conner"
- 18 "Bruce Wayne"
- 19 "Jericho"
- 20 "Atonement"
- 21 "Fallen"
- 22 "E.L._.O."
- 23 "Faux Hawk"
- 24 "Nightwing"
- 25 "Barbara Gordon"
- 26 "Red Hood"
- 27 "Hank & Dove"
- 28 "Blackfire"

```
29 "Lazarus"  
30 "Lady Vic"  
31 "51%"  
32 "Home"  
33 "Souls"  
34 "Troubled Water"  
35 "The Call Is Coming from Inside the House"  
36 "Prodigal"  
37 "Purple Rain"  
38 "Lex Luthor"  
39 "Mother Mayhem"  
40 "Jinx"  
41 "Super Super Mart"  
42 "Inside Man"  
43 "Brother Blood"  
44 "Caul's Folly"  
45 "Dick & Carol & Ted & Kory"  
46 "Dude, Where's My Gar?"  
47 "Game Over"  
48 "Project Starfire"  
49 "Titans Forever"
```

Recuerda que esto lo puedes utilizar en un *DataFrame*, mandar un diccionario, o enviar a cualquier estructura de Python que necesites trabajar posteriormente.

| Material adicional

British Council. (s.f.). *Irregular verbs* [Verbos irregulares]. LearnEnglish. Recuperado de <https://learnenglish.britishcouncil.org/grammar/english-grammar-reference/irregular-verbs>

Python Package Index. (s. f.). *pip documentation* [Documentación de *pip*]. Recuperado de <https://pip.pypa.io/en/latest/>

Wikipedia. (s. f.). Anexo: Películas de Walt Disney Pictures. Recuperado de https://es.wikipedia.org/wiki/Anexo:Pel%C3%ADculas_de_Walt_Disney_Pictures

Wikipedia. (s. f.). *List of Super Bowl champions*. Recuperado de https://en.wikipedia.org/wiki/List_of_Super_Bowl_champions

Wikipedia. (s.f.). *Titans (2018 TV series)* [Titanes (serie de televisión de 2018)]. Recuperado el 30 de marzo de 2025, de [https://en.wikipedia.org/wiki/Titans_\(2018_TV_series\)](https://en.wikipedia.org/wiki/Titans_(2018_TV_series))

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 30 de marzo del 2025 y su disponibilidad depende del autor.

| Bibliografía

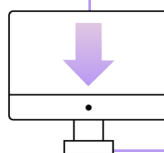
Richardson, L. (s.f). Beautiful Soup Documentation. Crummy. Recuperado el 30 de marzo de 2025, de <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

W3C. (s. f.). *World Wide Web Consortium (W3C)* [Consortio de la World Wide Web]. Recuperado el 30 de marzo de 2025, de <https://www.w3.org/>

Wikipedia. (s. f.). Anexo: Películas de Walt Disney Pictures. Recuperado el 30 de marzo de 2025, de https://es.wikipedia.org/wiki/Anexo:Pel%C3%ADculas_de_Walt_Disney_Pictures

Wikipedia. (s. f.). *Titans (2018 TV series)* [Titans (serie de televisión 2018)]. Recuperado el 30 de marzo de 2025, de [https://en.wikipedia.org/wiki/Titans_\(2018_TV_series\)](https://en.wikipedia.org/wiki/Titans_(2018_TV_series))

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 30 de marzo del 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.